# CMU-CS 462: Software Measurement and Analysis

## Spring 2025-2026

**Measuring Internal Product Attributes: Structural Complexity**

**Nguyen Duc Man**
**E: mannd@duytan.edu.vn**
**T: 0904235945**



1

## Use Case Point /3

- Use Case Point Estimation has its origin in the work by Gustav Karner (1993)
- Process:
  1. Identify and weight Actors (→ UAW)
  2. Identify and weight Use Cases (→ UUCW)
  3. Calculate Unadjusted Use Case Points (→ UUCP = UAW + UUCW)
  4. Calculate Value Adjustment Factor (→ VAF)
  5. Calculate (Adjusted) Use Case Points
     → UCP = UUCP * VAF

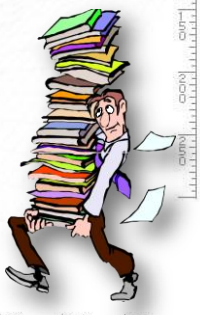


2

DUY TÂN UNIVERSITY

## Contents

1. Software structural measurement
2. Control-flow structure
3. Structural complexity: cyclomatic complexity
4. Data flow and data structure attributes
5. Architectural measurement





3

DUY TÂN UNIVERSITY

## Thảo luận theo cặp

1. **What are the key attributes of software structure, and why are they important in measuring complexity?**
2. **How does Cyclomatic Complexity help in understanding the complexity of a program? What are its advantages and limitations?**
3. **Discuss the differences between Basic Control Structures (BCSs) and Advanced Control Structures. How do they impact software complexity?**
4. **How do Control Flow Graphs (CFGs) help in visualizing software complexity? Provide examples of how CFGs can be used in analysis.**
5. **Why is there a trade-off between Control Flow Complexity and Data Structure Complexity? Provide examples to illustrate this trade-off.**
6. **How can measuring software complexity influence software development and maintenance? Provide real-world scenarios.**

4

DUY TÂN UNIVERSITY

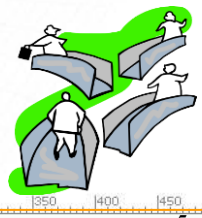
## Problem Statement

- How complex is the following program?

```
1: read x,y,z;
2: type =“scalene”;
3: if (x == y or x == z or y == z) type =“isosceles”;
4: if (x == y and x == z) type =“equilateral”;
5: if (x >= y+z or y >= x+z or z >= x+y) type =“not a triangle”;
6: if (x <= 0 or y <= 0 or z <= 0) type =“bad inputs”;
7: print type;
```

- Is there a way to measure it?

**It has something to do with program structure (branches, nesting) & flow of data**

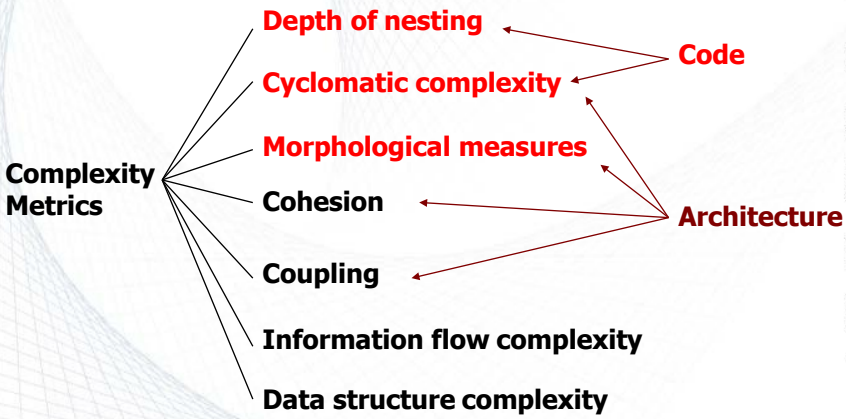


5

5

DUY TÂN UNIVERSITY

## Software Complexity Metrics




6

6

DUY TÂN UNIVERSITY

## How to Represent Program Structure?

Software structure can have 3 attributes:

- **Control-flow structure:** Sequence of execution of instructions of the program.
- **Data flow:** Keeping track of data as it is created or handled by the program.
- **Data structure:** The organization of data itself independent of the program.

7

7

DUY TÂN UNIVERSITY

## Goal & Questions …

- Q1: How to represent "structure" of a program?
- A1: Control-flow diagram
- Q2: How to define "complexity" in terms of the structure?
- A2: Cyclomatic complexity; depth of nesting

8

8

DUY TÂN UNIVERSITY

## Basic Control Structure /1

- **Basic Control Structures (BCSs)** are set of essential control-flow mechanisms used for building the logical structure of the program.
- BCS types:
  - **Sequence:** e.g., a list of instructions with no other BCSs involved.
  - **Selection:** e.g., *if … then … else*.
  - **Iteration:** e.g., *do … while* ; *for … to … do*.

9

9

DUY TÂN UNIVERSITY

## Basic Control Structure /2

- There are other types of BCSs, (may be called advanced BCSs), such as:
  - **Procedure/function/agent call**
  - **Recursion (self-call)**
  - **Interrupt**
  - **Concurrence**

10

10

DUY TÂN UNIVERSITY

## Control Flow Graph (CFG) /1

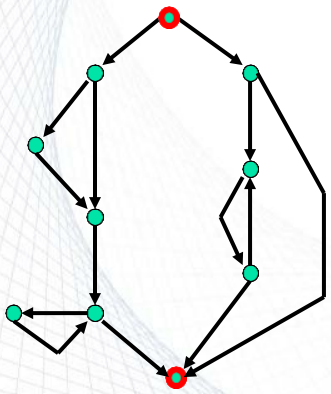
- Control flow structure is usually modeled by a directed graph (di-graph)

  CFG = {N, A}
- Each node *n* in the set of nodes (N) corresponds to a program statement.
- Each directed arc (or directed edge) *a* in the set of arcs (A) indicates flow of control from one statement of program to another.
  - **Procedure nodes:** nodes with out-degree 1.
  - **Predicate nodes:** nodes with out-degree other than 1 and 0.
  - **Start node:** nodes with in-degree 0.
  - **Terminal (end) nodes:** nodes with out-degree 0.

11

DUY TÂN UNIVERSITY

## Control Flow Graph (CFG) /2



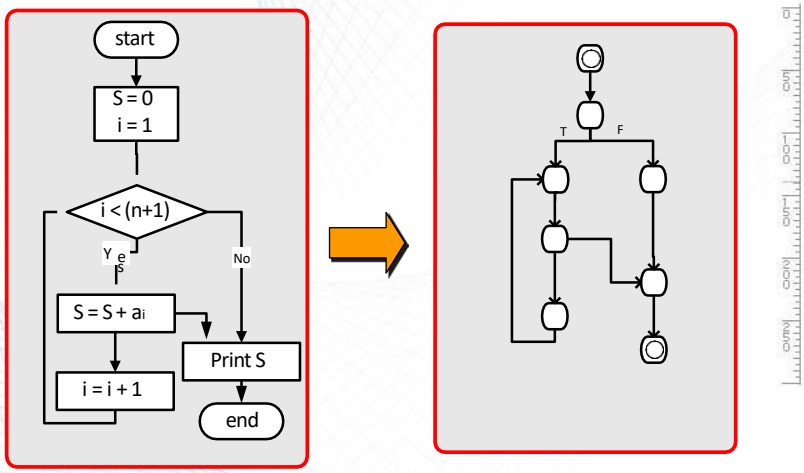
Definition [FeR97]:

- A flowgraph is a directed graph in which two nodes, the start node and the stop node, obey special properties: the stop node has out-degree zero, and the start node has in-degree zero. Every node lies on some path from the start node to the stop node.

12

DUY TÂN UNIVERSITY

## CFG Example 1



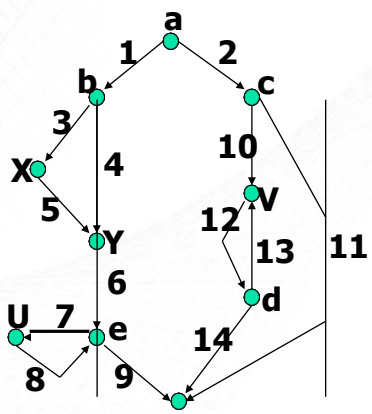

11

13

DUY TÂN UNIVERSITY

## CFG Example 2

```
if a then
   if b then X
   Y
   while e do U
else
   if c then
      repeat V until d
   endif
endif
```




14

DUY TÂN UNIVERSITY

## Control Flow Graph /2

- The control flow graph CFG = {N, A} model for a program does not explicitly indicate how the control is transferred. The **finite-state machine (FSM)** model for CFG does.

$M = \{N, \Sigma, \delta, n_0, F\}$

$N$ set of nodes; $\Sigma$ set of input symbols (arcs)

$\delta$ transition function

$\delta(p, a) = q$ ; $p, q \in N$ and $a \in \Sigma$

$n_0 \in$ N starting node and $F \subseteq N$ set of terminal node(s)

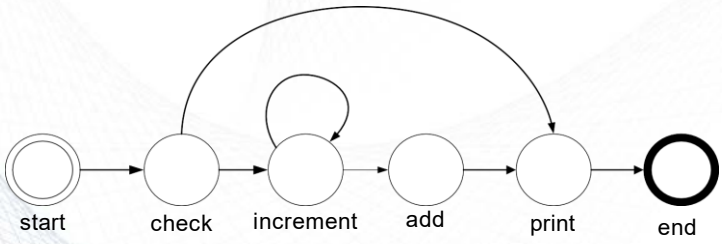
15

15

DUY TÂN UNIVERSITY

## Example: FSM Model

- Finite state machine model for the increment and add example

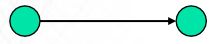
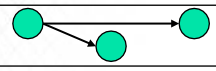
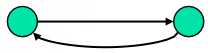
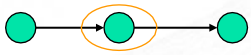
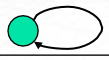
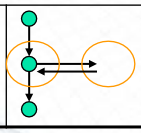
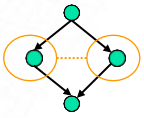



16

16

DUY TÂN UNIVERSITY

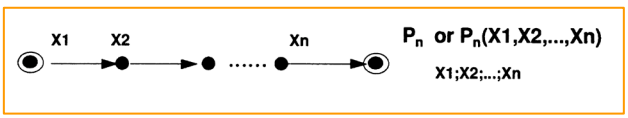
## Control Flow Graph /3

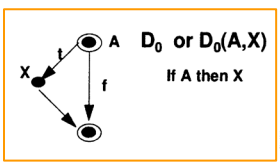
| | |
|---|---|
| Sequence |  |
| Selection |  |
| Iteration |  |
| Procedure/ function call |  |
| Recursion |  |
| Interrupt  | Concurrence  |

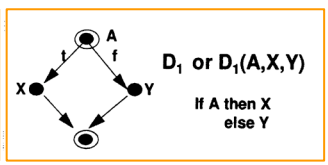


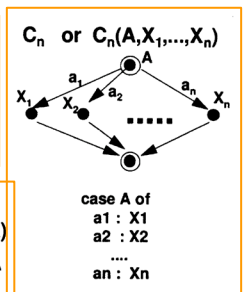


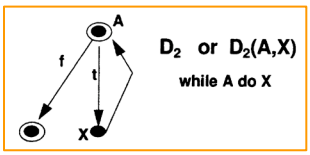
## Common CFG Program Models

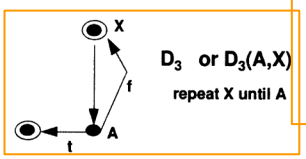


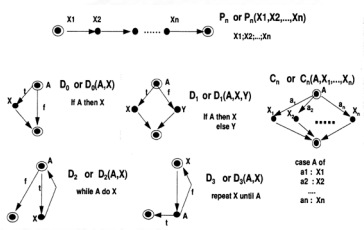



DUY TÂN UNIVERSITY

# Prime Flow Graphs

- Prime flow graphs are flow graphs that cannot be decomposed non-trivially by sequencing and nesting.
- **Examples (according to [FeP97]):**
  - $P_n$ (sequence of n statements)
  - $D_0$ (if-condition)
  - $D_1$ (if-then-else-branching)
  - $D_2$ (while-loop)
  - $D_3$ (repeat-loop)
  - $C_n$ (case)

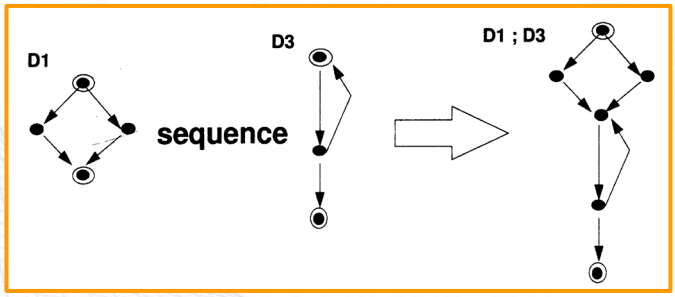



19

DUY TÂN UNIVERSITY

# Sequencing & Nesting /1

- Let **F1** and **F2** be two flowgraphs. Then, the sequence of **F1** and **F2**, (shown by **F1; F2**) is a flowgraph formed by merging the terminal node of **F1** with the start node of **F2**.



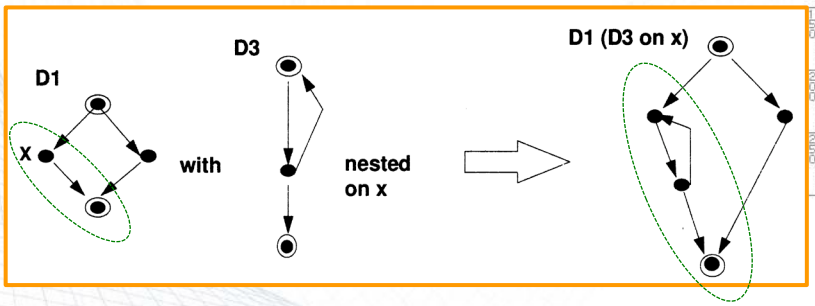

20

20

DUY TÂN UNIVERSITY

## Sequencing & Nesting /2

- Let **F1** and **F2** be two flowgraphs. Then, the nesting of **F2** onto **F1** at *x*, shown by **F1(F2)** is a flowgraph formed from **F1** by replacing the arc from *x* with the whole of **F2**.








DUY TÂN UNIVERSITY

## Cyclomatic Complexity

- A program's complexity can be measured by the cyclomatic number of the program flowgraph.
- The cyclomatic number can be calculated in 2 different ways:
  - Flowgraph-based
  - Code-based

DUY TÂN UNIVERSITY

## Cyclomatic Complexity /1

- For a program with the program flowgraph *G*, the cyclomatic complexity *v(G)* is measured as:

  $v(G) = e - n + 2p$

  - *e* : number of edges
    - Representing branches and cycles
  - *n* : number of nodes
    - Representing block of sequential code
  - *p* : number of connected components
    - For a single component, *p=1*

23



DUY TÂN UNIVERSITY

## Cyclomatic Complexity /2

- For a program with the program flowgraph *G*, the cyclomatic complexity *v(G)* is measured as:

  $v(G) = 1 + d$

  - *d* : number of predicate nodes (i.e., nodes with out-degree other than 1)
    - d represents number of loops in the graph
    - or number of decision points in the program

i.e., The complexity of primes depends only on the predicates (decision points or BCSs) in them.

24

DUY TÂN UNIVERSITY

## Cyclomatic Complexity: Example

$v(G) = e - n + 2p$

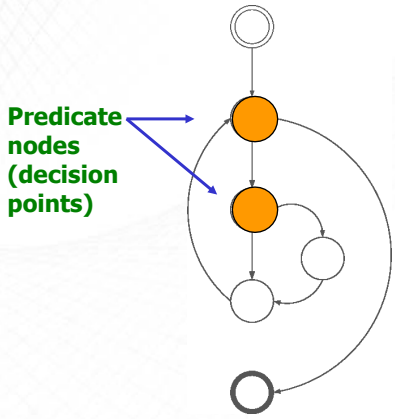
$v(G) = 7 - 6 + 2 \times 1$

$v(G) = 3$

Or

$v(G) = 1 + d$

$v(G) = 1 + 2 = 3$




25

25

DUY TÂN UNIVERSITY

## Example: Code Based

```
#include <stdio.h>
main()
{
int a ;
scanf ("%d", &a);
  if ( a >= 10 )
       if ( a <  20 )        printf   ("10 < a< 20 %d\n ,  a);
                                                  "
    else                      printf ("a >= 20     %d\n ,  a);
                                                  "
else                          printf ("a <= 10     %d\n ,  a);
                                                  "
}
```

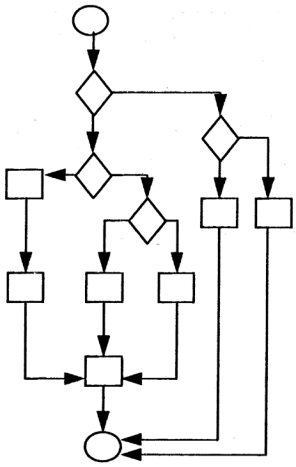
$v(G) = 1+2 = 3$

26

26

DUY TÂN UNIVERSITY

# Example: Graph Based



$v(G) = 16 - 13 + 2 = 5$

*or*

$v(G) = 4 + 1 = 5$

27



DUY TÂN UNIVERSITY

# Example 1

- Determine cyclomatic complexity for the following Java program:

**v = 1+d**

**v = 1+6 = 7**

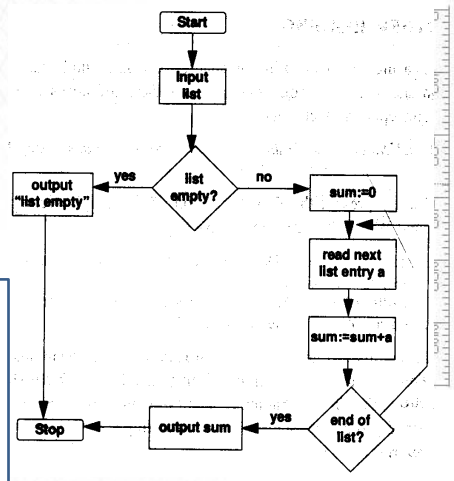
```
1.  import java.util.*;
2.  public class CalendarTest
03. {
04. public static void main(String[] args)
05. {
6.   // construct d as current date
7.   GregorianCalendar d = new GregorianCalendar();
8.   int today = d.get(Calendar.DAY_OF_MONTH);
9.   int month = d.get(Calendar.MONTH);
10. // set d to start date of the month
11. d.set(Calendar.DAY_OF_MONTH, 1);
12. int weekday = d.get(Calendar.DAY_OF_WEEK);
13. // print heading
14. System.out.println("Sun Mon Tue Wed Thu Fri Sat");
15. // indent first line of calendar
16. for (int i = Calendar.SUNDAY; i < weekday; i++ )
17. System.out.print("   ");
18. do
19. {
20. // print day
21. int day = d.get(Calendar.DAY_OF_MONTH);
22. if (day < 10) System.out.print(" ");
23. System.out.print(day);
24. // mark current day with *
25. if (day == today)
26. System.out.print("* ");
27. else
28. System.out.print(" ");
29. // start a new line after every Saturday
30. if (weekday == Calendar.SATURDAY)
31. System.out.println();
32. // advance d to the next day
33. d.add(Calendar.DAY_OF_MONTH, 1);
34. weekday = d.get(Calendar.DAY_OF_WEEK);
35. }
36. while (d.get(Calendar.MONTH) == month);
37. // the loop exits when d is day 1 of the next month
38. // print final end of line if necessary
39. if (weekday != Calendar.SUNDAY)
40. System.out.println();
41. }
42. }
```

28

DUY TÂN UNIVERSITY

# Example 2

- Determine cyclomatic complexity for the following flow diagram:




29



DUY TÂN UNIVERSITY

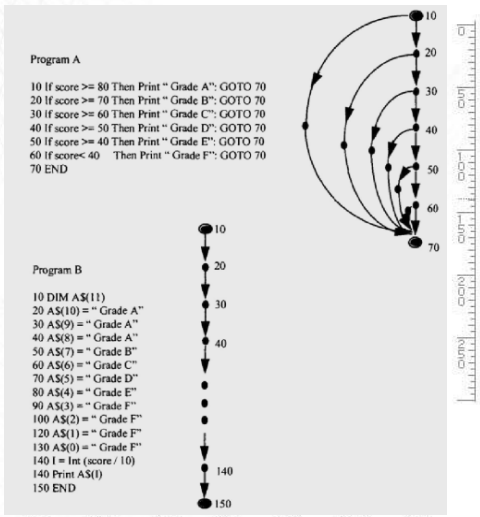
# Example 3A

- Two functionally equivalent programs that are coded differently
- Calculate cyclomatic complexity for both

$V_A = 7$

$V_B = 1$

**There is always a trade-off between control-flow and data structure. Programs with higher cyclomatic complexity usually have less complex data structure. Apparently program B requires more effort that program A.**




30

DUY TÂN UNIVERSITY

## Cyclomatic Complexity: Critics

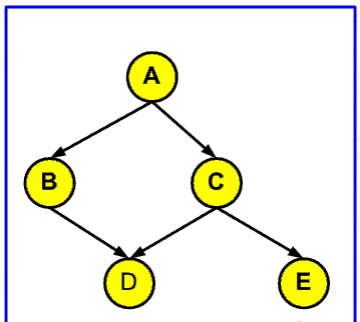
- **Advantages:**
  - Objective measurement of complexity
- **Disadvantages:**
  - Can only be used at the component level
  - Two programs having the same cyclomatic complexity number may need different programming effort
  - Same requirements can be programmed in various ways with different cyclomatic complexities
  - Requires complete design or code visibility

31

DUY TÂN UNIVERSITY

## Software Architecture /1

- A software system can be represented by a graph,
  S = {N, R}
- Each node *n* in the set of nodes (N) corresponds to a subsystem.
- Each edge *r* in the set of relations (R) indicates a relation (e.g., function call, etc.) between two subsystems.




39

32

DUY TÂN UNIVERSITY

## Data Structure Measurement

- There is always a trade-off between control-flow and data structure.
- Programs with higher cyclomatic complexity usually have less complex data structure.
- A simple example for data-structure measurement:

| | |
|---|---|
| Integers | $C_1 = n_i \times 1$ |
| Strings | $C_2 = n_s \times 2$ |
| Arrays | $C_3 = n_a \times 2 \times size\ of\ array$ |
| Total | $C = C_1 + C_2 + C_3$ |

77

33

DUY TÂN UNIVERSITY

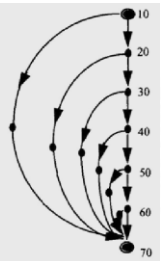
## Example 3B

- Calculate date structure complexity for Program A and B

**There is always a trade-off between control-flow and data structure. Programs with higher cyclomatic complexity usually have less complex data structure. Apparently program B requires more effort that program A.**

Program A

```
10 If score >= 80 Then Print " Grade A": GOTO 70
20 If score >= 70 Then Print " Grade B": GOTO 70
30 If score >= 60 Then Print " Grade C": GOTO 70
40 If score >= 50 Then Print " Grade D": GOTO 70
50 If score >= 40 Then Print " Grade E": GOTO 70
60 If score< 40    Then Print " Grade F": GOTO 70
70 END
```




Program B

```
10 DIM A$(11)
20 A$(10) = " Grade A"
30 A$(9) = " Grade A"
40 A$(8) = " Grade A"
50 A$(7) = " Grade B"
60 A$(6) = " Grade C"
70 A$(5) = " Grade D"
80 A$(4) = " Grade E"
90 A$(3) = " Grade F"
100 A$(2) = " Grade F"
120 A$(1) = " Grade F"
130 A$(0) = " Grade F"
140 I = Int (score / 10)
140 Print A$(I)
150 END
```


78

34

DUY TÂN UNIVERSITY

## Exercise: Switch statement

```
     ...  // other stuff
A1    switch(wParam)
      {
A2      case VK_UP:
          MoveUp(hWnd);
          break;
A3      case VK_DOWN:
          MoveDown(hWnd);
          break;
A4      case VK_LEFT:
          MoveLeft(hWnd);
          break;
A5      case VK_RIGHT:
          MoveRight(hWnd);
          break;
A6    }
      ...  // other stuff
```

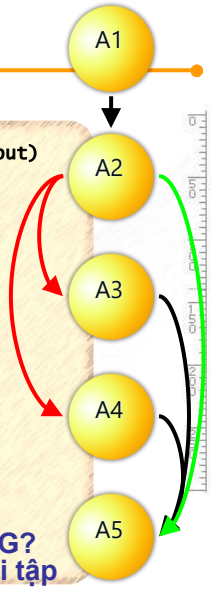
Work in group of 3, draw CFG?
Thảo luận nhóm 3 để làm bài tập

35

DUY TÂN UNIVERSITY

## Exercise : Exception handling

```
A1  public bool ConvertToInteger(string input, ref int output)
    {
        try
        {
A2          output= Convert.ToInt32(input);
        }
A3      catch (OverflowException, e)
        {
            MessageBox.Show (e.Message);
            return false;
        }
A4      catch (FormatException, e)
        {
            MessageBox.Show (e.Message);
            return false;
        }
        return true;
A5  }
```




Work in group of 3, draw CFG?
Thảo luận nhóm 3 để làm bài tập

36

DUY TÂN UNIVERSITY

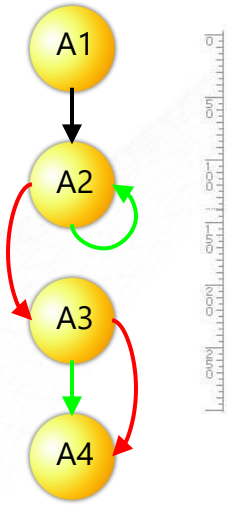
## Example: Control flow

Control flow graph

Aid to trace execution paths through a program / function.

```
A1  public static int CharIndex(string str, char c)
    {
        int i = 0;
        int retVal = 0;
        char[] s  = str.ToCharArray();

A2      while ((i < s.Length) && (s[i] != c))
        {
            i++;
        }
A3      if (i < s.Length)
        {
            retVal = i;
        }
        return retVal;
A4  }
```




37

DUY TÂN UNIVERSITY

## Practice: Decision coverage

Create control flow diagram, define tests

```
internal static decial GetCommission(int locks, int stocks, int barrels)
{
  decimal sales = GetTotalSales(locks, stocks, barrels);
  decimal commission = 100;
  if (CommissonIsDue(locks, stocks, barrels))  {
    if (sales > 1800)  {
      commission += (800 * Bonus15);
      commission += ((sales - 1800m) * Bonus20);
    }
    else if (sales > 1000)  {
      commission += ((sales - 1000m) * Bonus15);
    }
    else  {
      commission = (sales * standardCommission);
    }
   }

    return commission.ToString();
}
```

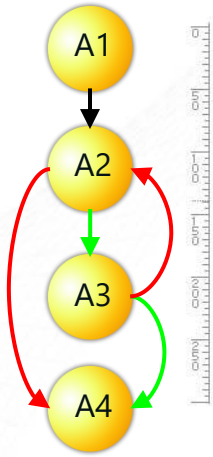
38

DUY TÂN UNIVERSITY

## Loops

```
A1  private static bool IsThirtyDayMonth(int month)
    {
        int[] shortMonth = new int[4] { 4, 6, 9, 11 };

A2      for (int i = 0; i < shortMonth.Length; i++)
        {
A3          if ( month == shortMonth[i])
            {
                return true;
            }
        }
        return false;
A4  }
```

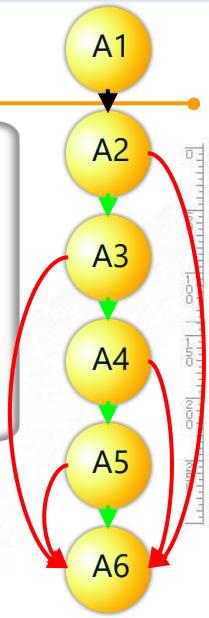



39

DUY TÂN UNIVERSITY

## Example: Condition testing

```
A1  private bool IsInValidDate (int month, int day, int
year)
    {
                A2                  A3
        if ((year == 1582 && month == 10) &&
              A4            A5
            (day >= 5 && day <= 14))
        {
            return true;
        }
        return false;
A6  }
```




Assumes short-circuiting
Does not evaluate relational operators

40

DUY TÂN UNIVERSITY

# Practice: Condition testing

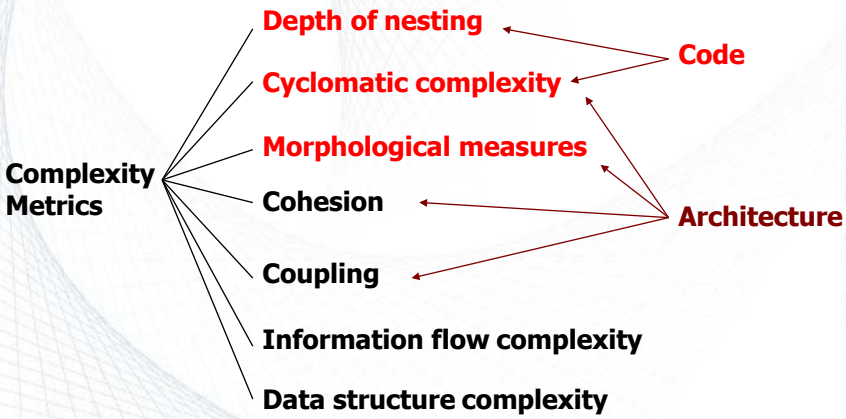
Assume valid integer inputs for params a, b, and c

```
A0          private void GetTriangle (int a, int b, int c)
            {
A1, A2, A3      if (!(a + b > c) || !(b + c > a) || !(a + c > b))
                {
                    // Error
                }
A4, A5          else if ((a == b) && (b == c))
                {
                    // Equilateral;
                }
A6, A7, A8      else if ((a == b) || (b == c) || (a == c))
                {
                    // Isoceles;
                }
                else
                {
                    // Scalene;
                }
A9          }
```

41

DUY TÂN UNIVERSITY

# Conclusion: Complexity Metrics




79

42

DUY TÂN UNIVERSITY

## Summary

**Watch the following video and answer the below question.**

- **https://www.youtube.com/watch?v=eZmBDMEguzc**

**Q. What are the methods to find the cyclomatic complexity ?**





DUY TÂN UNIVERSITY

## References


- Park, R. E. (1992). *Software size measurement: A framework for counting source statements* (No. CMU/SEI/92-TR-20). Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst. (available at URL: http://www.sei.cmu.edu/pub/documents/92.reports/pdf/tr20.92.pdf )